AI GTM Engine

System Design & Architecture Document

Architecture baseline: benkogerrie/ai-gtm-engine, branch b004a-search-layer
Immutable commit: e28cd3d13a924a098c88eb5793826c0fa5e2a90c
Version: 1.0 | **Prepared:** 30 October 2026 | **Status:** implementation-aligned design specification

Scope note. This document anchors implementation claims to the requested GitHub branch, not the later main branch. The branch contains a Next.js foundation, relational domain/evidence model, DeepSeek provider and Brave search provider. An integrated GTM workflow, graph traversal engine, real contact enrichment, production messaging, and the bonus services are **not implemented in this branch**. The detailed designs below are target architecture and acceptance criteria, not assertions that these features are already running. The earlier attached seven-page buildathon PDF could not be re-opened in this session; the repository and its documented roadmap are the verifiable technical source. Validate final submission wording against the event brief.

1. Executive architecture decision

The AI GTM Engine turns a founder's plain-language B2B product description into an editable ideal customer profile (ICP), searches for named buyers and competitors, collects attributable evidence, builds a relationship graph, scores accounts, prepares evidence-grounded outreach for human approval, and exposes an auditable pipeline. The system is a **decision-support and preparation tool**, not an autonomous outreach sender.

Architectural pattern: one Next.js 16 / TypeScript application; explicit service interfaces for search and LLM; Zod domain contracts; PostgreSQL/Drizzle as the transactional system of record; a derived graph projection for multi-hop analysis; a staged workflow orchestrator; immutable audit events; optional metering and agent-operations integrations. Prefer a modular monolith during the buildathon over separate microservices.

Non-negotiable invariants

A search result is a candidate, not a verified fact. VERIFIED requires a real source, retrieval timestamp, same-tenant attribution, and a trustworthy acceptance decision; link presence alone does not establish truth.

Synthetic/mock sources never enter the real evidence ledger as Brave evidence.

No outbound message is sent without a same-organization, message-specific human approval. The demo runs in dry-run mode by default.

Organization and actor identity are resolved server-side, never trusted from caller-supplied organization IDs or headers.

Every persisted change is auditable; audit rows cannot be edited or deleted.

Existing code is independent of COR and must stay so.

2. Existing implementation inventory

Application — **Implemented on requested branch:** Next.js 16 App Router, React 19, TypeScript strict, Tailwind 4, route handlers, Vitest, GitHub Actions; **Missing / future target:** Full GTM pages and orchestrated UX

Domain and storage — **Implemented on requested branch:** Zod entities; Drizzle PostgreSQL schema; migrations; PGlite fallback; transactional repository and audit events; **Missing / future target:** Durable hosted PostgreSQL configured for production

Authentication — **Implemented on requested branch:** Server-configured bearer token mapped to APP_ORG_ID / actor; public health API; **Missing / future target:** User sessions, granular roles and scalable multi-tenant auth

Language model — **Implemented on requested branch:** LlmClient interface; DeepSeek chat + schema-validated JSON; mock client; usage metadata; **Missing / future target:** ICP and reasoning workflows consuming the adapter

Search — **Implemented on requested branch:** SearchClient interface; Brave HTTP client; URL normalization, dedupe, rate limiting, timeouts/retries; mock-source barrier; **Missing / future target:** Query planner, entity extraction and persistence orchestration

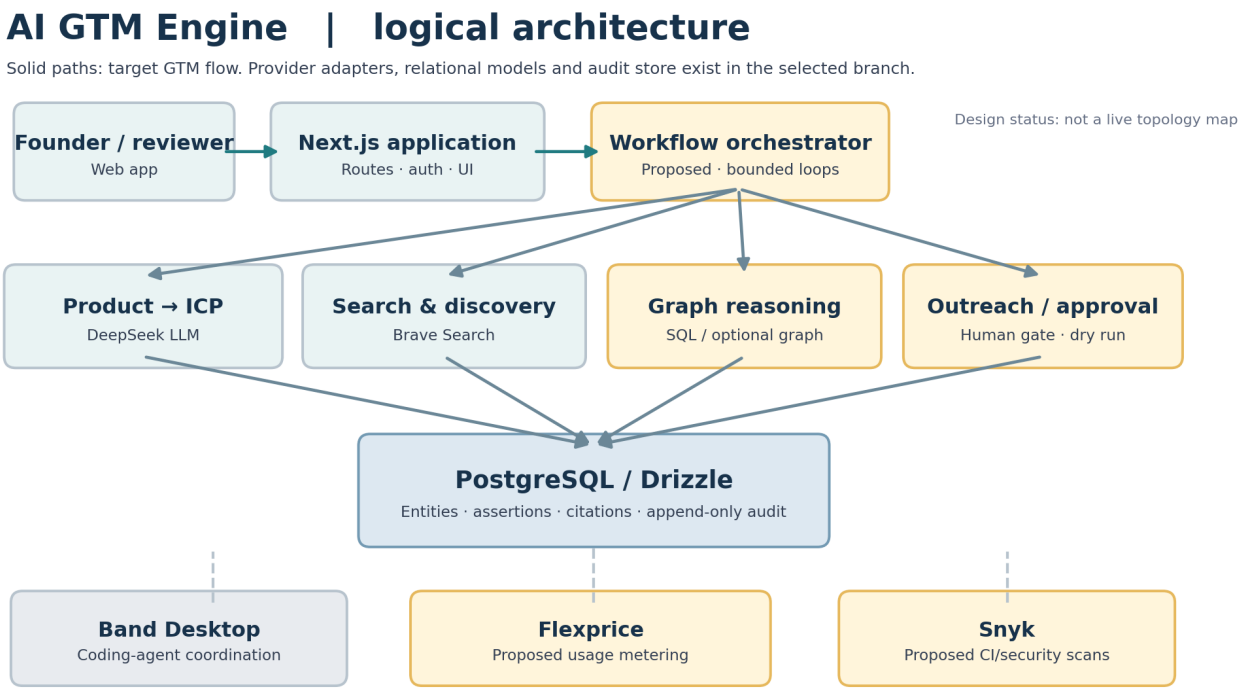
Graph — **Implemented on requested branch:** relationship_edges table; six allowed relationship types; **Missing / future target:** Graph materializer, traversal/ranking, visualization; optional Neo4j/Kùzu

Outreach — **Implemented on requested branch:** Persisted status model, approval record and domain guards; **Missing / future target:** Message drafting, approval UI and delivery adapter

Bonus — **Implemented on requested branch:** Placeholder configuration for Flexprice; generic CI; **Missing / future target:** Band Desktop workflow, Flexprice event export, Snyk scanning on this branch

Relevant code: src/domain, src/db/schema.ts, src/db/repository.ts, src/lib/llm, src/lib/search.X

3. System context and component architecture



Deployment: Vercel Preview/Next.js Node runtime; DATABASE_URL selects PostgreSQL via pg. Without it, the code chooses PGlite. Local CLI defaults to .pglite/; serverless previews without PostgreSQL are ephemeral. The target demo must configure durable PostgreSQL before presenting persistent pipeline behavior.

Trust boundaries: untrusted user text and web snippets enter the planner/extractor; provider keys remain server-side; validated structured objects cross into the repository; outbound contact is a separate privileged operation following approval. Agent development messages in Band are outside the GTM application's runtime and data trust boundary.

4. Logical data model and provenance

The existing relational entities are Organization, Product, ICP, Company, Person, Competitor, Signal, Lead, Outreach, OutreachApproval, Citation, FieldAssertion, AuditEvent, and RelationshipEdge. Entities are organization-scoped and carry UUID identifiers and timestamps. FieldAssertion connects a particular value (entityType, entityId, fieldName, fieldValue) to its verification status; many-to-many field_assertion_citations links claims to sources.

Product — **Key attributes:** Description, features, capabilities, constraints, pricing; **Responsibility:** Seller's offer and query seed

ICP — **Key attributes:** Industries, size/revenue bands, personas, triggers, confidence map; **Responsibility:** Hypothesis of best-fit buyers

Company — **Key attributes:** Domain, geography, industry, headcount, fit/intent, tier; **Responsibility:** Deduplicated buyer account

Person — **Key attributes:** Company, role, contact status, email, LinkedIn; **Responsibility:** Potential decision maker; no assumed verified contact

Competitor — **Key attributes:** Product, direct/indirect/alternative, positioning, tiers; **Responsibility:** Substitutes and competition

Signal — **Key attributes:** Company, signal type, occurrence, strength; **Responsibility:** Buying or negative trigger

Citation — **Key attributes:** Source URL, retrievedAt, type, quote, SHA-256 content hash, extractor; **Responsibility:** Provenance record

FieldAssertion — **Key attributes:** Field value, verification status, asserted-by/time; **Responsibility:** Claim lifecycle

RelationshipEdge — **Key attributes:** Source/target type and IDs, typed edge; **Responsibility:** Graph-compatible relation

Outreach/Approval — **Key attributes:** Status, dryRun, channel, approval actor/time; **Responsibility:** Controlled communication

AuditEvent — **Key attributes:** Actor, action, entity, before/after, timestamps; **Responsibility:** Append-only change history

Verification state semantics (implemented enum): UNVERIFIED = unknown/unsubstantiated; INFERRED = model judgement without qualifying source; VERIFIED = externally supported and accepted claim; REJECTED = contested or invalidated. A Citation is necessary to transition to VERIFIED but insufficient without source relevance and a reviewer/trusted-extractor check. A search-result snippet does not prove a person's email, company size or vendor deployment.

Recommended additional provenance fields (not on requested branch): source trust tier, observed date versus retrieved date, statement-to-quote alignment confidence, extraction prompt/model version, verification method and reviewer, freshness expiry, runId, raw result reference, and contradiction/supersession link. Preserve prior assertions rather than silently overwriting them.

5. Query logic and search strategy

5.1 Query pipeline

Inputs: Product.description, relevant features/capabilities, constraints, approved or draft ICP, target country, persona titles, trigger events and explicit exclusion domains. In the requested branch, only the search adapter exists; the query planner described here is **proposed**.

Normalize product category, customer pain, vocabulary, industries, geography, decision-maker roles and negative constraints with Zod-validated structured output.

Expand synonyms, common abbreviations and adjacent terms while preserving exact brand and sector names; record why each expansion was made.

Generate 3–6 intent-specific queries per discovery run: direct buyer evidence, buying triggers, public personnel evidence, competitor differentiation and counter-evidence. Exclude listicles and generic vendor directories.

Execute Brave searches through SearchClient.search(query, { count, offset, country, freshness }); respect the existing client minimum 1,000 ms inter-request gap, its timeouts and retry policy.

Normalize/de-duplicate URLs with the existing URL utility; de-duplicate organizations by normalized domain + canonical identity rather than URL alone.

Extract only organizations and claims supported by returned snippets or follow-up page fetches, with an exact matched source reference. Classify buyer, competitor/vendor, or media/research; do not turn article publishers into prospects.

Corroborate and challenge a claimed fact with independent sources where risk warrants. Separate fact verification from inferred fit and intent.

Persist source records and assertions transactionally; score and rank only after validation; record run, query, budget and exclusion reasons.

5.2 Example deterministic query families

Sample seller: a fictional B2B application for hospital shift scheduling. Queries are templates, not evidence of a particular buyer.

```
Q1 buyer:    (hospital OR "health system") ("nurse scheduling" OR "staffing operations") site:.org -"top 10"
-directory
Q2 signal:   (hospital OR "health system") ("staffing expansion" OR "schedule optimization") announcement
Q3 hiring:   (hospital OR "health system") ("nurse scheduler" OR "workforce operations") hiring
Q4 persona:  "hospital name" ("VP Nursing" OR "Workforce Management Director")
Q5 rival:    ("nurse scheduling software" OR "workforce rostering") competitor pricing
Q6 negative: "hospital name" ("hiring freeze" OR "budget cuts" OR "vendor contract")
```

Constraints: query text must not assert that a prospect uses a system or has a budget; site: constraints are only applied where useful; avoid assuming all hospital websites are .org. Search provider options are bounded by code: count 1–20 and offset 0–9. country and freshness are optional. Deep pagination is not available in the current adapter.

5.3 Retrieval and quality scoring — proposed

```
query_plan = buildQueries(product, icp, filters)
for query in query_plan[0:6]:
  hits = searchClient.search(query, {count: 10, country: targetCountry})
```

```
  eligible = hits.filter(isRealSearchEvidence)
  candidates += normalizeAndClassify(eligible)
organizations = dedupeByCanonicalIdentity(candidates)
for organization in organizations:
  claims = extractClaimsWithExactSourceLinks(organization)
  claims = rejectUnsupportedOrCrossTenantClaims(claims)
  evidence = verifyRelevantClaims(claims)
  persistWithAudit(organization, evidence)
rank(organizations, deterministicScore(evidence, icp))
```

Quality gates: no fabricated company; cited URL exists in the actual returned results; at least one relevant source per presented live lead; normalized, working HTTPS URL; snippet/published-date freshness clearly distinguished from retrieval time; original source URL retained. Webpage text is treated as untrusted input (prompt-injection risk); it cannot change tool permissions or approval policy.

6. Graph ontology and multi-hop reasoning

6.1 Graph storage strategy

Implemented: PostgreSQL relationship_edges with orgId, fromEntityType, fromEntityId, toEntityType, toEntityId, and edgeType; this is a relational stand-in, not a working graph traversal subsystem. **Target:** PostgreSQL remains authoritative for entities, citations, permissions and audit. A graph materializer emits idempotent typed nodes/edges to Neo4j or Kùzu only if multi-hop needs justify the extra moving part. A Postgres recursive CTE is acceptable for the buildathon MVP.

6.2 Node ontology

Organization — **Primary identity:** orgId; **Proposed graph properties:** Tenant boundary; never traverse across tenants

Product — **Primary identity:** product.id; **Proposed graph properties:** Description, category, capabilities

ICP — **Primary identity:** icp.id; **Proposed graph properties:** Industries, personas, status, confidence

Company — **Primary identity:** canonical company UUID; **Proposed graph properties:** Domain, industry, fit tier, active signals

Person — **Primary identity:** person.id; **Proposed graph properties:** Role and verification, scoped employer

Competitor — **Primary identity:** competitor.id; **Proposed graph properties:** Category and positioning

Signal — **Primary identity:** signal.id; **Proposed graph properties:** Type, occurredAt, relevance, polarity

Citation/Assertion — **Primary identity:** citation and assertion IDs; **Proposed graph properties:** Provenance, evidence status and source

Lead/Outreach — **Primary identity:** lead and message UUIDs; **Proposed graph properties:** Pipeline stage, approval/dry-run state

6.3 Typed edge ontology

The following six edges are implemented as allowed RelationshipEdgeType enum values: WORKS_AT, COMPETES_WITH, MATCHED_TO, CONTACTED_VIA, REPLIED_TO, PREVIOUSLY_EMPLOYED_AT.

WORKS_AT — **Recommended endpoints:** Person → Company; **Semantics / evidence rule:** Current employment, must be cited or marked unverified

PREVIOUSLY_EMPLOYED_AT — **Recommended endpoints:** Person → Company; **Semantics / evidence rule:** Historical affiliation, with dates if available

COMPETES_WITH — **Recommended endpoints:** Company/Competitor → Company/Competitor; **Semantics / evidence rule:** Direct, indirect or alternative rivalry; support with cited positioning

MATCHED_TO — **Recommended endpoints:** Company → ICP; **Semantics / evidence rule:** Computed fit; mark as INFERRED and include score version

CONTACTED_VIA — **Recommended endpoints:** Lead/Person → Outreach; **Semantics / evidence rule:** Message association; only SENT indicates actual delivery

REPLIED_TO — **Recommended endpoints:** Person → Outreach; **Semantics / evidence rule:** Observed reply or human assertion; do not infer from approval

Proposed additional edge types: OFFERS (Company→Product), HAS_SIGNAL (Company→Signal), SUPPORTED_BY (Assertion→Citation), HAS_CONTACT (Lead→Person), ALTERNATIVE_TO (Product→Competitor), and TARGETS (Product→ICP). None should be described as implemented. Enforce an allowlist of (edgeType, fromType,

toType), existing same-tenant endpoints, non-duplicate keys and evidence metadata; current schema does not provide those typed endpoint constraints by itself.

6.4 Queries and explanations

Example business question: “Which high-fit health systems show a relevant staffing signal and have a verified operations decision maker?”

```
// Proposed Neo4j projection, not runnable against current PostgreSQL schema
MATCH (c:Company {orgId:$orgId})-[m:MATCHED_TO]->(i:ICP {id:$icpId})
MATCH (c)-[:HAS_SIGNAL]->(s:Signal)
MATCH (p:Person)-[:WORKS_AT]->(c)
WHERE m.fitScore >= 75 AND s.polarity = 'POSITIVE'
  AND p.contactStatus = 'VERIFIED'
RETURN c.name, m.fitScore, s.summary, p.fullName, p.title
ORDER BY m.fitScore DESC LIMIT 20;
```

For MVP, materialize equivalent lookups in SQL using companies, signals, people, icps, field_assertions and relationship_edges with orgId checks. The explainable answer should show a path of node IDs, edge types, citation IDs, unknowns and inference scores. A path can support a recommendation but is not proof that the company intends to buy.

7. Reasoning loops and agent orchestration

A bounded, explicit state machine is preferable to an open-ended recursive agent. Each stage has typed inputs/outputs, budget, failure branch, retry policy, human checkpoint and an audit event. The requested branch provides the LLM adapter, not this orchestrator.

7.1 Loop A — Product → ICP

INTAKE_RECEIVED → EXTRACT_PRODUCT → INFER_ICP → VALIDATE_SCHEMA → HUMAN_REVIEW → ICP_APPROVED / REJECTED.

DeepSeek LlmClient.json(schema, messages) enforces JSON parsing and Zod; on bad JSON/schema a single repair turn is attempted, then the call errors. Proposed ICP validation tests sector constraints, persona specificity, unsupported assumptions and confidence bounds [0,1]. Do not label inferred ICP fields verified.

7.2 Loop B — Discovery / evidence

PLAN_QUERIES → SEARCH → EXTRACT → NORMALIZE → VERIFY → NEGATIVE_CHECK → SCORE → COMMIT → REVIEW_RESULTS.

Use deterministic queries and limited search budget. The **verification loop** identifies source-claim mismatch, conflicting employment dates, unsupported financial claims and expired facts, then retries with a new targeted query only when the likely information gain justifies the cost. End with an explicit unknown rather than fabricating evidence.

7.3 Loop C — Graph / hypothesis

For each account, propose hypotheses such as “new hiring activity may imply staffing change”; traverse relevant graph paths; find one corroborating and one disconfirming signal; label path-level confidence; compute fit and intent separately; generate a short explanation with links to citations. Loop terminates on evidence sufficiency, query budget or maximum depth (recommended depth ≤3 for MVP). No path alone proves purchase intent.

7.4 Loop D — Outreach / human gate

SELECT_VERIFIED_FACTS → DRAFT → FACT_CHECK → HUMAN_EDIT → PENDING_APPROVAL → APPROVED / REJECTED → DRY_RUN_COMPLETED.

Messages may reference only source-backed claims actually passed into the prompt; model-generated URLs and names must be checked against the allowed source set. Suppress outreach for opted-out or bounced persons. The domain supports SENT, but real delivery must remain disabled until a trusted provider and explicit approval are implemented.

7.5 Retry, budget and stop conditions

DeepSeek HTTP — **Default policy now:** Max 3 attempts; retry 429/5xx; 30 s per attempt; exponential backoff; **Recommended workflow policy:** Global token/cost budget, one schema repair, no unbounded agent recursion

Brave HTTP — **Default policy now:** Max 3 attempts; retry 429/5xx; honor Retry-After; 15 s; ≥1 s request spacing; **Recommended workflow policy:** ≤6 search queries/run, cache/dedupe, domain exclusions

Evidence — **Default policy now:** Synthetic result rejected; VERIFIED requires citation with URL and date; **Recommended workflow policy:** Add independent source/claim acceptance and freshness checks

Workflow — **Default policy now:** No orchestrator in branch; **Recommended workflow policy:** Persist checkpoint, idempotency key, run status and failure reason

Messaging — **Default policy now:** Dry-run defaults true; approval guard exists; **Recommended workflow policy:** Human-only approval, irreversible dry-run state, no real send in demo

Recommended orchestration object (new): GtmRun {id, orgId, productId, icpId, state, inputHash, modelVersion, queryPlan, spentTokens, spentSearches, createdAt, finishedAt, errorCode}. Persist each transition; make retries resume from a checkpoint, not re-create duplicate prospects.

8. Fit, intent and evidence-based ranking

The current Company model stores fitScore, intentScore (0–100) and fitTier (A/B/C) but the branch does not implement the ranking algorithm. Proposed transparent baseline:

fit = 0.30·industry + 0.20·size + 0.20·problem_match + 0.15·region + 0.15·persona_reach (each feature normalized to 0–100). intent = 0.45·positive_signal + 0.20·signal_recency + 0.15·signal_strength + 0.20·negative_signal_adjustment. Calculate priority = 0.70·fit + 0.30·intent, then apply exclusion/uncertainty penalties. These are **design weights**, not repository behavior or validated predictive coefficients.

Default display tiers (proposal): A ≥80, B 60–79, C <60; suppress hard exclusions even if numeric score is high. Always show reason codes, evidence freshness, positive/negative signals and an independent confidence/coverage score. Avoid interpreting a numeric fit score as a probability of conversion.

9. API and interface contracts

Implemented server routes: public GET /api/health, GET /api/llm/health, GET /api/search/health; authenticated GET/POST /api/products, GET /api/companies/[id], POST /api/citations, GET /api/audit. Unauthorized data calls return 401; invalid payloads and domain violations return 400. Server uses Authorization: Bearer $APP_API_TOKEN; APP_ORG_ID and actor identity come from environment. Health endpoints indicate configuration only, not live provider connectivity.

Proposed routes (not in requested branch): POST /api/icp/infer, POST /api/icp/{id}/approve, POST /api/discovery/runs, GET /api/discovery/runs/{id}, GET /api/graph/paths, POST /api/outreach/drafts, POST /api/outreach/{id}/approval, POST /api/outreach/{id}/dry-run, GET /api/dashboard, GET /api/usage. Each must derive tenant context server-side and reject cross-tenant IDs.

Provider interfaces: SearchClient.search(query, options): Promise<SearchResult[]>; normalized result includes title, url, snippet, optional publishedAt, source, retrievedAt. LlmClient.chat and .json(schema, messages) return content/typed object and model/token/latency metadata. Both require a real key unless mock fallback is explicitly enabled. Do not silently enable mocks for live user-facing evidence.

10. Bonus service integrations

10.1 Band Desktop — development operations, not GTM runtime

Status on branch: not integrated. Band Desktop is a desktop collaboration layer for local coding agents and shared rooms; it can connect Cursor through the Agent Client Protocol. It does **not** execute the GTM application's buyer research or become a customer-facing feature just by being present. Its legitimate role is coordinating specification → implementation → independent review while preserving human decisions.

Proposed workflow: create one Band room for the ai-gtm-engine repository; attach this architecture Markdown as the room plan; assign an implementation agent (Cursor), test agent and reviewer; require PR links, commit hashes, CI results and reviewer decisions in the shared task board; route merge/deployment questions to the owner. Reconcile Notion IDs with GitHub PR numbers; never grant agent blanket permission to merge to main or disclose secrets. The user may continue using GrokBot as coordinator, with Band as an optional team operations surface.

Acceptance evidence: room screenshot showing agents and tasks; architecture plan attached; one reproducible task handoff; PR URL and CI evidence; human sign-off. Official docs: https://docs.band.ai/band-desktop and https://docs.band.ai/connect/coding-agents.

10.2 Flexprice — runtime usage metering and monetization

Status on branch: FLEXPRICE_API_KEY is merely a commented placeholder in .env.example. No Flexprice SDK, request or event exporter exists.

Proposed integration: a server-only UsageMeter interface receives successful, uniquely identified usage events from the orchestrator; an outbox table stores eventId, orgId, runId, eventName, units, provider, timestamp, delivery state. Export asynchronously to Flexprice, retrying safely with stable idempotency IDs. Start with brave.search_request,

llm.input_tokens, llm.output_tokens, lead.enriched, outreach.dry_run. Charge only defined billable events; do not invent dollar costs when model prices are unverified. LlmUsageRecord already provides token counts and latency as the metering source. A browser must never call Flexprice with the API key.

```
{
  "event_name": "llm.output_tokens",
  "external_customer_id": "tenant_external_id",
  "event_id": "gtm-run-uuid:deepseek-completion:001",
  "timestamp": "2026-10-30T20:00:00Z",
  "properties": { "tokens": "825", "provider": "deepseek", "model": "configured-model" }
}
```

This payload is **illustrative** and must be validated against the deployed Flexprice API/version, customer mapping, metric definitions and required headers. The API documents POST /v1/events, x-api-key and external_customer_id; see https://github.com/flexprice/flexprice/wiki/Flexprice-API-Reference:-Endpoint-Specifications-and-Request-Examples and https://docs.flexprice.io. Acceptance: event visible once per run, tenant separated, failed exports recover without duplication, usage totals match the application's ledger.X

10.3 Snyk — supply-chain and source-code security

Status on branch: its .github/workflows/ci.yml runs install, typecheck, lint, unit tests and build; it does not run Snyk. There is a separate PR #2 for Snyk work, but its contents are **not** part of the requested branch. Do not claim a security gate passes without its own CI evidence.

Proposed integration: on pull requests scan dependencies (snyk test) and source (snyk code test) using a secret token stored in GitHub Actions, then upload/report findings. Run production dependency risk and exploitability review separately from dev-only tool issues. Fail on new critical or actionable high vulnerabilities under the chosen policy, with time-limited documented exceptions. Acceptance: reproducible scan report, no exposed token, branch protection/merge rule and remediation register. Official reference: https://docs.snyk.io/developer-tools/snyk-cli/commands/test and https://docs.snyk.io/developer-tools/snyk-cli/commands/code-test.

11. Security, safety and compliance boundaries

Tenant isolation: every read/write and graph query scoped by server-resolved orgId; verify both endpoints of every relationship edge and every linked citation belong to that tenant. The current server token maps one configured org/actor; it is not full multi-user authentication.

Secrets: DeepSeek/Brave/Postgres/Flexprice/Snyk credentials are server-side environment variables or CI secrets. Public health responses must not contain secrets. Redact provider error bodies before displaying them to end users.

Prompt injection: returned web text can only be treated as data. Never allow search snippets to override system prompts, authorize messaging, request secrets or trigger unrelated URLs.

Evidence integrity: distinguish citations, field assertions, inferences and rejected contradictions. Hashes detect content changes, not truth. Cross-tenant citation and object references are forbidden.

Contact data: use legitimate sources and permissioned enrichment; respect opt-outs, reasonable retention and regional outreach law. MX validity alone is not email deliverability or consent.

Outreach: human-specific approval binds exact message content/version; editing an approved message invalidates approval. Require idempotent dispatch and a final kill switch before any actual send is enabled.

Durability: Postgres for hosted demo; migrations are version-controlled; backup/restore plan for production. PGlite serverless preview without DATABASE_URL loses state.

Audit: repository withAudit writes an event in the same DB transaction as mutation. Migration 0001_cto_review_guards.sql adds a trigger blocking audit UPDATE/DELETE. Also protect DB credentials and administrative permissions outside app code.

12. Observability and evaluation

Capture per-run query count, Brave rate-limit/429 count, DeepSeek tokens and latency, model/plan version, lead yield, deduplication rate, source coverage, factual corrections, unsupported-claim rejection rate, approval rate, dry-run completions, and provider spend/credits. Collect structured logs with runId, orgId (hashed in external telemetry as needed) and stage, but exclude raw credentials and unnecessary personal data. Metrics should distinguish LIVE from MOCK.

Quality tests: (1) exact-source extraction and no invented URL; (2) cross-tenant citation/edge authorization; (3) duplicate organizations across multiple sources; (4) conflicting employment/signal dates; (5) mock evidence rejection; (6) no send without human approval; (7) dry-run never sets sentAt; (8) provider 429/retry behavior; (9) failed Flexprice export retried exactly once logically; (10) end-to-end live run with verifiable sources and database persistence.

Demo contract: the reviewer can follow one product from intake through ICP, discover at least three real and distinct buyer organizations with clickable cited evidence, see a graph path and its uncertainty, inspect a source-grounded outreach draft, approve and complete dry-run only, and verify dashboard state plus usage/audit. Where a feature remains unbuilt, display a clear stub/status rather than fabricated successful output.

13. Design decisions and unresolved gaps

ADR-01 — **Decision / gap:** Modular monolith; no premature microservices; **Rationale and action:** Fast build, single deploy and transaction boundaries

ADR-02 — **Decision / gap:** PostgreSQL is system of record; **Rationale and action:** Audit, evidence and tenant-safe atomic writes

ADR-03 — **Decision / gap:** Graph is projection / SQL traversal first; **Rationale and action:** Avoid graph-storage inconsistency until multi-hop value is proven

ADR-04 — **Decision / gap:** Typed provider interfaces; **Rationale and action:** Replace DeepSeek or Brave without rewriting workflows

ADR-05 — **Decision / gap:** Human-approved dry-run only; **Rationale and action:** Prevent unwanted contact in hackathon demo

GAP-01 — **Decision / gap:** No orchestrator or graph traversal on requested branch; **Rationale and action:** Build bounded stages, run state, idempotency, cited path explanations

GAP-02 — **Decision / gap:** Citation existence alone is not independent fact verification; **Rationale and action:** Add claim/source relevance and acceptance workflow

GAP-03 — **Decision / gap:** No durable DB on default serverless preview; **Rationale and action:** Configure hosted PostgreSQL and test migrations on clean instance

GAP-04 — **Decision / gap:** Current auth uses one environment token/org; **Rationale and action:** Add sessions/RBAC for multiuser deployment

GAP-05 — **Decision / gap:** Relationship edge endpoint validation is not yet graph-grade; **Rationale and action:** Add endpoint schemas, same-org checks and dedupe constraints

GAP-06 — **Decision / gap:** Band/Flexprice/Snyk not in this branch; **Rationale and action:** Implement separately, collect independently verifiable proof

GAP-07 — **Decision / gap:** Current DeepSeek default model id may be account-dependent; **Rationale and action:** Set model explicitly and verify provider model availability

GAP-08 — **Decision / gap:** Repository main has progressed beyond target branch; **Rationale and action:** Reconcile this baseline with latest code before final submission

14. Implementation sequence and exit criteria

Foundation (completed on branch): domain, evidence ledger, DeepSeek/Brave adapters, tests; confirm migration and authorization tests.

Product and query planning: product intake → typed ICP; editable approval, deterministic 3–6 query plan; record versions and budget.

Discovery and evidence: named buyers, competitor filtering, source-backed assertions, organization dedupe, negative research and rationale.

Graph and scoring: same-tenant edge validation, derived graph read model, multi-hop query, explainable path and calibrated scoring.

Outreach and pipeline: cited drafts, explicit approval, dry-run, no real send, audited Kanban/status derivation.

Bonus integrations: Band task room + evidence, Flexprice usage outbox/export, Snyk CI findings and exception policy.

Submission hardening: persistent PostgreSQL, production-like smoke run, source audit, threat check, screenshots, README, architecture diagram, 2–3-minute demo narrative, and repo visibility check.

Release gate: run pnpm typecheck, pnpm lint, pnpm test, pnpm build; then a gated live smoke with configured credentials and persistent PostgreSQL. Pass only if no mock source is presented as verified, no unapproved contact is sent, and all demo claims can be traced to code and evidence.

15. References and traceability

Immutable source anchor:
https://github.com/benkogerrie/ai-gtm-engine/tree/e28cd3d13a924a098c88eb5793826c0fa5e2a90cX

Repository documents at requested commit:

Domain and verification: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/docs/domain-model.mdX

DeepSeek adapter: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/docs/llm.mdX

Brave adapter: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/docs/search.mdX

Domain rules: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/src/domain/rules.tsX

Audit and persistence: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/src/db/repository.tsX

Provider CI: https://github.com/benkogerrie/ai-gtm-engine/blob/e28cd3d/.github/workflows/ci.ymlX

External vendor documentation: Band (https://docs.band.ai/band-desktop); Flexprice (https://docs.flexprice.io and API wiki above); Snyk (https://docs.snyk.io/developer-tools/overview); Brave (https://api-dashboard.search.brave.com/documentation/services/web-search); DeepSeek (https://api-docs.deepseek.com/quick_start/pricing).X

Document classification: architecture baseline and target specification. “Implemented” is limited to the selected branch commit. “Proposed” is not a claim of shipped functionality.